

PI

Pi 源码分析

从 Agent Loop 到可扩展 Agent Runtime

*源码调用链 · Session Tree · Context Compaction · Tool Runtime ·
Extension · SDK*

Source baseline: earendil-works/pi · commit 581d75a89cea21e50d6a26df840352f94427f633

编制日期: 2026-08-13

阅读目标 不是学习 Pi 的命令，而是从源码抽取一个 Agent Runtime 应该怎样分层、怎样处理工具、上下文、会话、扩展与生命周期。

0. 阅读说明与源码基线

本手册基于 Pi 当前源码快照进行结构化分析。为了区分事实与推论，正文使用两类表达：

- “源码事实”：直接对应仓库中的类型、函数、调用关系或注释。
- “工程解读”：基于源码结构提炼出的设计原则、trade-off 与自研建议。

层	主要路径	本文关注点
AI / Provider	packages/ai	统一模型、消息、流式接口与 provider 边界
Agent Core	packages/agent	Agent、runAgentLoop、Tool Runtime、Events、Queues
Coding Agent	packages/coding-agent/src/core	AgentSession、SessionManager、Compaction、Extensions、SDK
UI / Modes	packages/coding-agent/src/modes	只作为宿主层理解，不作为源码学习入口

建议的源码阅读顺序

- packages/agent/src/agent-loop.ts
- packages/agent/src/types.ts
- packages/agent/src/agent.ts
- packages/coding-agent/src/core/sdk.ts
- packages/coding-agent/src/core/agent-session.ts
- packages/coding-agent/src/core/session-manager.ts
- packages/coding-agent/src/core/compaction/compaction.ts
- packages/coding-agent/src/core/compaction/branch-summarization.ts
- packages/coding-agent/src/core/extensions/runner.ts
- packages/coding-agent/src/core/resource-loader.ts 与 system-prompt.ts
- 最后再看 interactive / RPC / TUI。

目录

1. 源码总架构：Core 与 Product Session 分离
 2. Agent Loop：两层循环模型
 3. Agent：Stateful Runtime Wrapper
 4. Message 与 Context Projection
 5. Streaming：partial state 如何进入 Runtime
 6. Event Model：把 Agent 变成 Event Producer
 7. Tool Runtime：从调用到结果的完整管线
 8. Permission / Policy Hook
 9. 并行工具与顺序语义
 10. AgentSession：Coding Agent 的 Application Service
 11. SDK Composition Root
 12. SessionManager：Append-only Tree
 13. Compaction：Context Projection，不删除历史
 14. Branch Summary 与 Context Memory
 15. AgentSessionRuntime：Session Replacement Lifecycle
 16. Extension：Mechanism 与 Policy 分离
 17. Provider Boundary 与 ModelRuntime
 18. 对照 learn-claude-code s01-s10
 19. 哪些设计值得复制，哪些不该照抄
 20. 自研 Pi-like Runtime 的推荐重构
- 附录 A：源码阅读 Checklist
- 附录 B：关键函数索引
- 附录 C：最小教学版 Runtime 伪代码

1. 源码总架构：Core 与 Product Session 分离

Pi 源码分层：产品层与执行内核分离

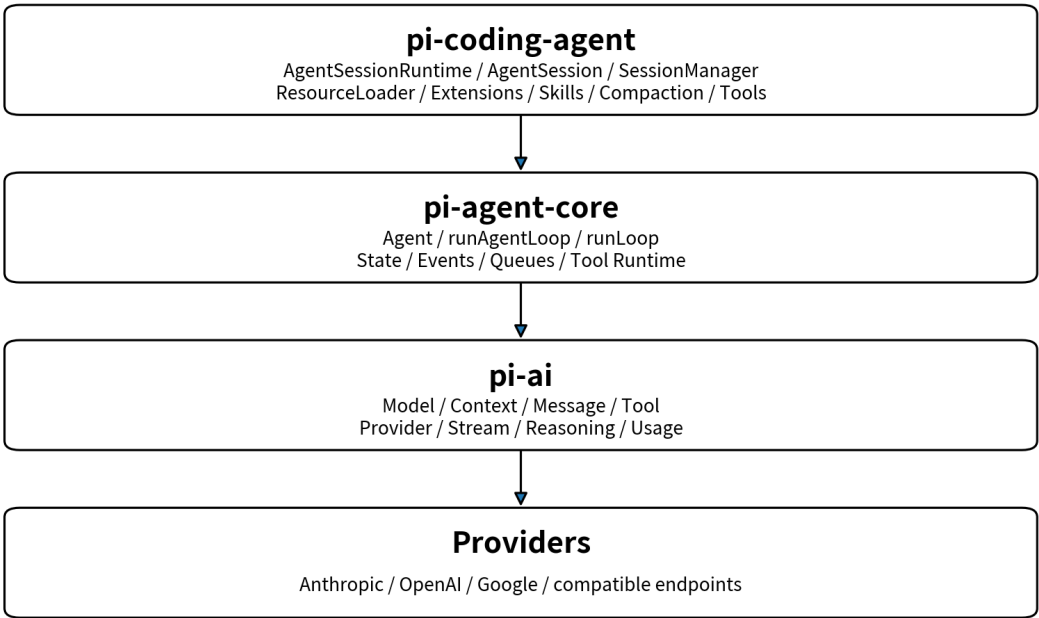


图 1 Pi 的主要源码分层：Provider、Agent Core 与 Coding Agent Orchestration 分离。

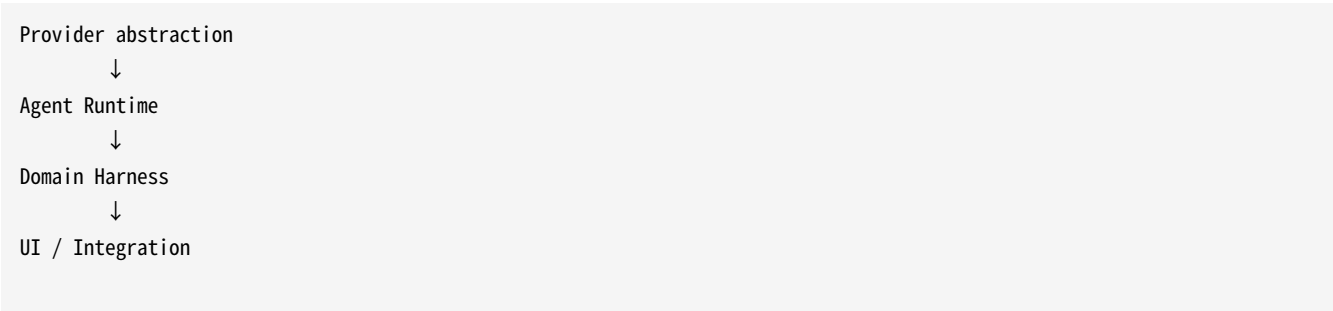
源码事实：`packages/agent` 提供通用 Agent 执行能力；`packages/coding-agent` 在其上增加 Coding Agent 所需的 session、资源加载、compaction、工具集合、扩展和交互模式。`AgentSession` 并不是低层循环本身，而是更高层的产品编排对象。

Know-why 如果把模型 SDK、Agent Loop、Session、CLI/TUI、Coding 规则写进同一个类，后续几乎无法把 Runtime 嵌入 IDE、RPC、Web 或自动化宿主。Pi 的分层让 “Agent Engine” 和 “Coding Product” 能独立演进。

四层职责

层	核心职责	不应该承担
pi-ai	模型/消息/流式/Provider 协议	Coding workflow、会话树
pi-agent-core	Agent Loop、state、events、tool execution、queues	AGENTS/Skills、Session JSONL、TUI
pi-coding-agent	Coding session、资源、compaction、extensions、tools	Provider SDK 细节
UI / modes	输入输出与用户交互	Agent 核心状态机

可迁移架构模式



2. Agent Loop：两层循环模型

runLoop()：两层循环，而不是单一 Tool Loop

Inner loop = 工具调用 + steering；Outer loop = follow-up continuation

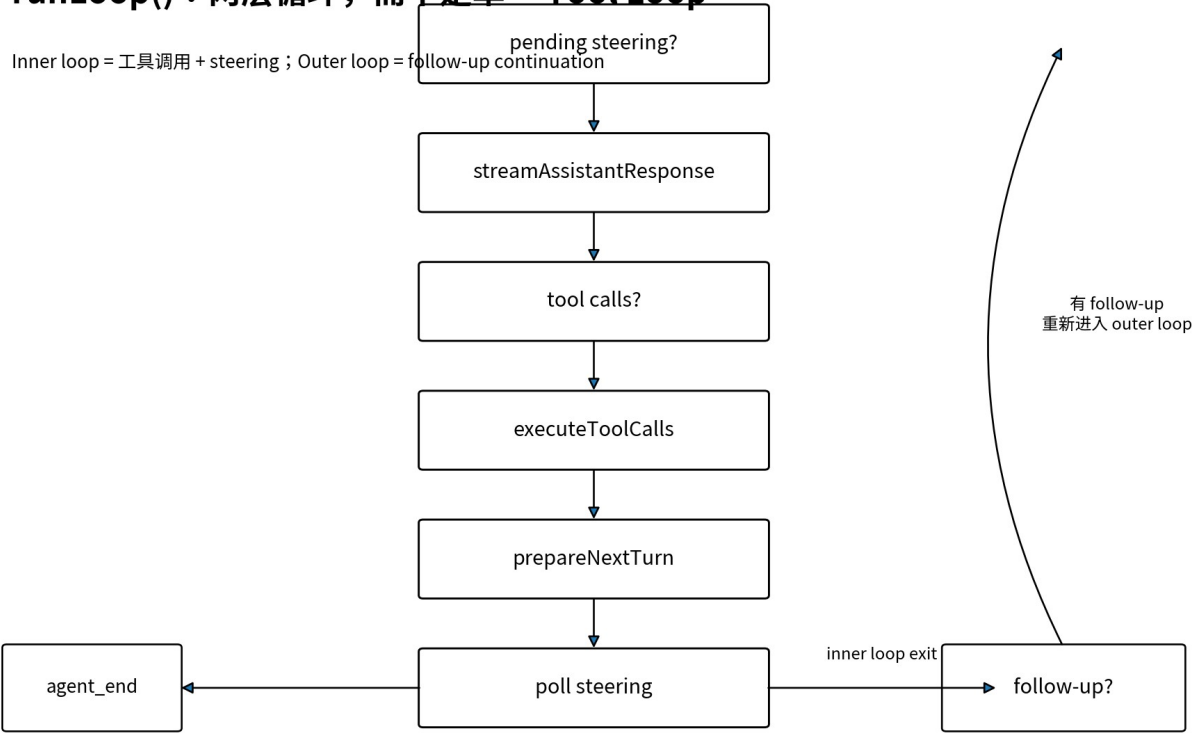


图 2 runLoop() 的核心：Inner loop 处理工具与 steering；Outer loop 处理 follow-up。

源码事实：`agentLoop()` / `agentLoopContinue()` 最终都进入共享的 `runLoop()`。`runLoop()` 维护 inner loop 与 outer loop 两层控制流：inner loop 在工具仍需继续或存在 steering 消息时运行；agent 原本准备结束时，outer loop 再查询 follow-up 队列。

为什么不是简单 while(tool_use)

一个生产 Agent 除了工具循环，还需要处理用户在执行期间追加的信息。如果把这些消息都当作新的 prompt，会导致并发 run；如果全部当作 abort，则人类无法低成本纠偏。Pi 通过 steer / follow-up 两种语义解决。

机制	注入时机	适合场景
----	------	------

steer	当前 assistant turn 结束后、下一次模型调用前	“别改数据库，换方案”
follow-up	Agent 本来准备完全停止之后	“完成后再补一个 changelog”

```
while true:
    # outer
    while tool_calls or steering: # inner
        inject_steering()
        msg = call_llm()
        results = execute_tools(msg)
        prepare_next_turn()
        steering = poll_steering()

    followups = poll_followups()
    if not followups:
        break
    steering = followups
```

Know-how 为 Agent 设计“消息队列语义”，而不是只有 prompt() 和 abort() 两个动作。队列语义本身应该属于 Runtime，而不是 UI workaround。

3. Agent: Stateful Runtime Wrapper

源码事实：`Agent` 是低层 loop 上方的 stateful wrapper。它持有 systemPrompt、model、thinkingLevel、tools、messages，以及 streamingMessage、pendingToolCalls、error 等运行态；同时持有 steeringQueue、followUpQueue 与 AbortController。

状态	用途
messages	Runtime transcript，而不等于最终发送给 Provider 的 context
tools	当前激活工具集合
streamingMessage	允许 UI / host 观察尚未完成的 assistant message
pendingToolCalls	运行态可观测性
activeRun / AbortController	阻止并发 prompt，并支持中止当前 run

并发约束

当 Agent 正在处理 prompt 时，再次调用 prompt 会被拒绝；新的用户意图必须进入 steer 或 follow-up 队列。这种限制避免两个 loop 同时修改同一个 transcript 与 tool state。

Know-how 单 Agent 实例最好只有一个 active run。并发任务应创建独立 Agent/Session，或者显式设计多执行分支，而不是允许两个 prompt() 竞争同一份 state。

4. Message 与 Context Projection

Pi 的一个关键抽象是：Runtime 内部的 AgentMessage 与 Provider 最终接收的 Message 不是同一概念。

```
AgentMessage[]
  ↓ transformContext()
AgentMessage[]
  ↓ convertToLlm()
Provider Message[]
  ↓
LLM
```

transformContext 与 convertToLlm 的职责区别

函数	输入/输出	语义
transformContext	AgentMessage[] → AgentMessage[]	Context Engineering: 压缩、裁剪、注入、extension transform
convertToLlm	AgentMessage[] → Provider Message[]	Protocol Projection: 过滤 runtime-only message, 转换自定义消息角色

工程解读：这是 “Storage/Runtime State ≠ Active Model Context” 的实现基础。后续 SessionManager、Compaction、custom entries 都依赖这个思想。

Know-why Memory 系统最常见的错误，是“存了什么就全部发给模型”。正确的抽象应该先完整保存状态，再按当前任务生成 context projection。

5. Streaming: partial state 如何进入 Runtime

`streamAssistantResponse()` 在模型开始返回时就把 partial assistant message 放入当前 context；text/thinking/toolcall delta 到来时，用新的 partial 替换最后一个 message；done/error 时再替换为 final message。

```
start    → append partial message
delta    → replace last runtime message
done     → replace with final message
message_end → persistence / listeners
```

这样做的好处是 UI、telemetry、extension 不需要自己维护第二套“正在生成中的消息”状态。Agent state 本身就是 source of truth。

风险点 partial state 进入共享状态后，所有 listener 都必须理解“尚未 final”的语义。对需要持久化或审计的系统，应只在明确的 message_end / finalized 事件上落盘。

6. Event Model: 把 Agent 变成 Event Producer

Pi 的运行过程被展开为细粒度生命周期事件。这个模型是把 UI、Session Persistence、Tracing、Eval、Extensions 从 Agent Loop 中解耦的关键。

```
agent_start
  turn_start
    message_start
    message_update ...
    message_end
    tool_execution_start
    tool_execution_update ...
    tool_execution_end
    message_start(toolResult)
    message_end(toolResult)
  turn_end
agent_end
```

事件消费者	能做什么
UI	增量渲染 text / thinking / tool progress
SessionManager	只在 finalized 消息完成时持久化
Telemetry	计算 latency、tool duration、tokens、failures
Eval	记录轨迹并做离线判定
Extensions	在生命周期边界注入行为

Know-how Runtime 应该 emit event，而不是直接 print、写数据库或更新 UI。宿主通过订阅事件承担这些副作用。

7. Tool Runtime：从调用到结果的完整管线

Tool Runtime Pipeline：权限、审计、结果变换都不必写进 Tool 本体

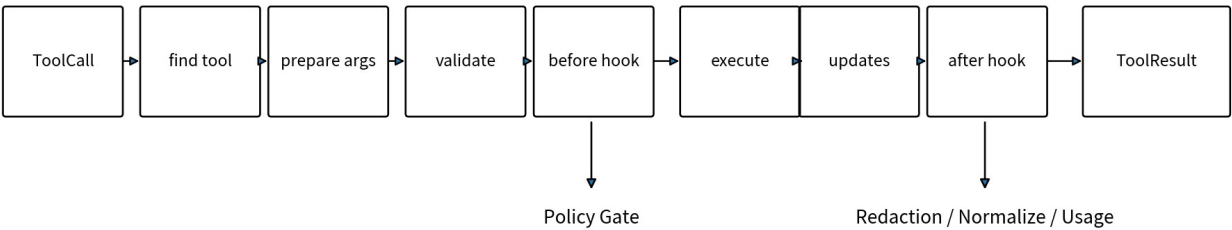


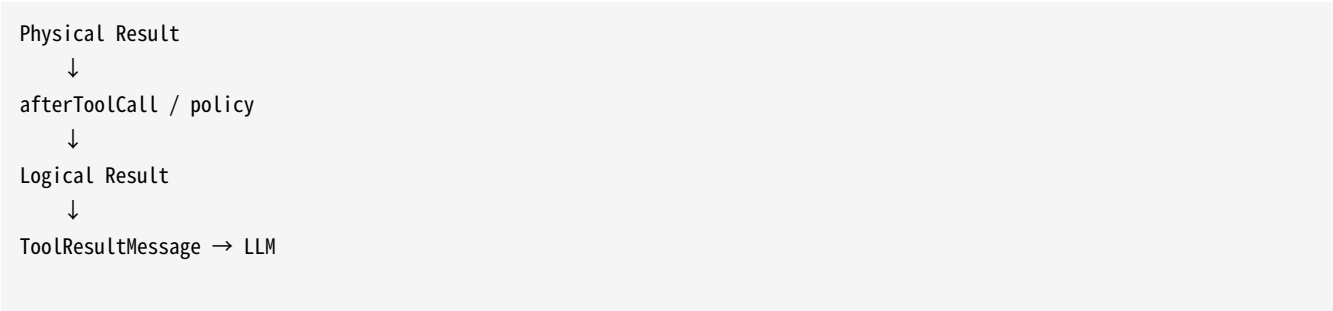
图 3 Tool execution pipeline。

源码事实：Tool execution 被拆成 preparation、execution、finalization 三阶段。工具参数可预处理，随后 schema validation；beforeToolCall 可阻止执行；工具执行期间可以发送 partial updates；afterToolCall 可以修改结果。

阶段	典型函数	职责
Prepare	prepareToolCall	查找工具、prepare args、validate、before hook
Execute	executePreparedToolCall	真正执行，并 emit progress update
Finalize	finalizeExecutedToolCall	after hook、normalize result、决定 error/terminate

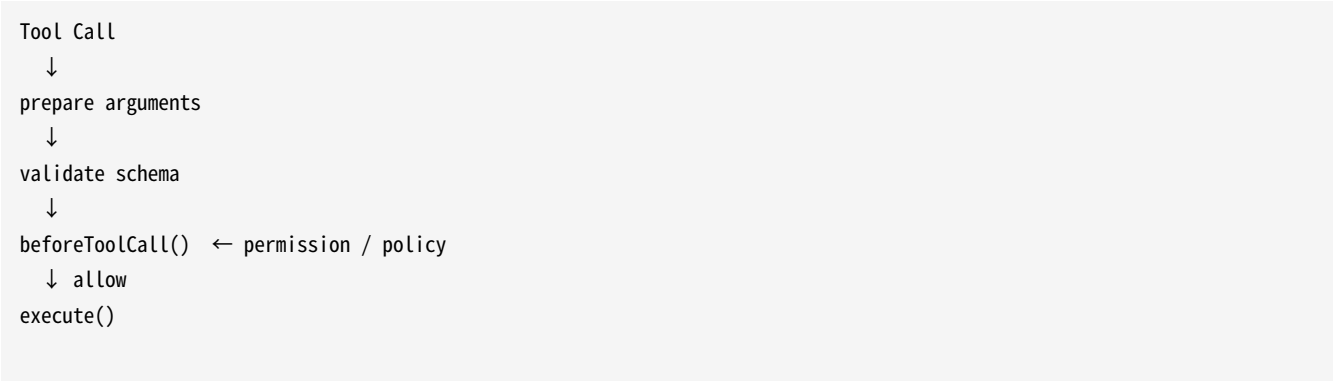
Tool Result 是逻辑结果，不只是物理结果

工具真实执行结果可以在 after hook 中被 redaction、normalization、usage enrichment 或转换。最终进入 LLM 的是 “logical result”。



8. Permission / Policy Hook

源码事实：beforeToolCall 可以 block，并给出 reason；还可以通过 terminate 结束当前工具批次。



推荐的权限分层

- 12. Gate 0: Sandbox / OS capability boundary。
- 13. Gate 1: Hard deny，例如绝对禁止的命令或资源。
- 14. Gate 2: Path / Resource policy。
- 15. Gate 3: Command / risk classification。
- 16. Gate 4: Human approval，仅用于需要用户决策的动作。
- 17. Gate 5: Execute。

Know-why Permission 不应该散落在 bash/read/write/edit 的实现里。把 policy 放在 tool pipeline 的统一 gate，才能让新工具自动继承相同安全规则。

截断 Tool Call 的 fail-closed 设计

如果 assistant 因输出长度限制被截断，Pi 不会执行该消息中的工具调用，而是统一产生错误结果要求模型重发。原因是：参数即使能被 salvage 成可解析 JSON，也可能语义上不完整。

原则 Parseable ≠ Complete。涉及外部副作用时，对不完整结构应该 fail closed，而不是猜测并执行。

9. 并行工具与顺序语义

默认 Tool execution 可以并行，但工具自身可以声明 sequential execution mode。只要本批次出现需要顺序执行的工具，运行时会采用顺序策略。

适合并行	通常应顺序
多个独立 read	write → edit 同一文件
多个搜索/查询	migration → verify
独立 API lookup	git add → commit

自研时可以进一步扩展 Tool Metadata

```
ToolMetadata {
  executionMode
  sideEffects
  idempotent
  riskLevel
  timeout
  concurrencyKey
}
```

Know-how Tool Schema 不只描述“怎么调用”，还应该描述“如何调度”和“副作用是什么”。这会直接影响并行、重试、权限与恢复。

10. AgentSession: Coding Agent 的 Application Service

AgentSession 是源码中真正把通用 Agent 变成 Coding Agent 的中枢。它负责将工具、扩展、session persistence、模型、compaction、retry、bash、资源等能力编排在一起。

Agent Core	AgentSession
通用 loop	Coding workflow orchestration
通用 before/after tool hook	把 hook 桥接到 ExtensionRunner
messages state	把 finalized message 落到 SessionManager
prepareNextTurn extension point	每轮刷新 systemPrompt/tools/model/thinking

late-bound next turn refresh

AgentSession 每轮都会刷新 system prompt、active tools、model 与 thinking level，避免长 run 中 Extension 或用户修改配置后继续使用旧 snapshot。

```
Conversation state = persistent
Execution configuration = late-bound per turn
```

Know-how 配置如果允许热切换，就不要在 run 开始时永久 snapshot。对工具、模型、系统提示等动态项，应在稳定的 turn boundary 重新解析。

11. SDK Composition Root

`createAgentSession()` 是理解整个 Coding Agent 如何组装起来的最佳入口之一。它承担 composition root：解析 cwd/agentDir，创建 ModelRuntime、SettingsManager、SessionManager、ResourceLoader，恢复 session/model/thinking，计算 active tools，构造 Agent，最后构造 AgentSession。

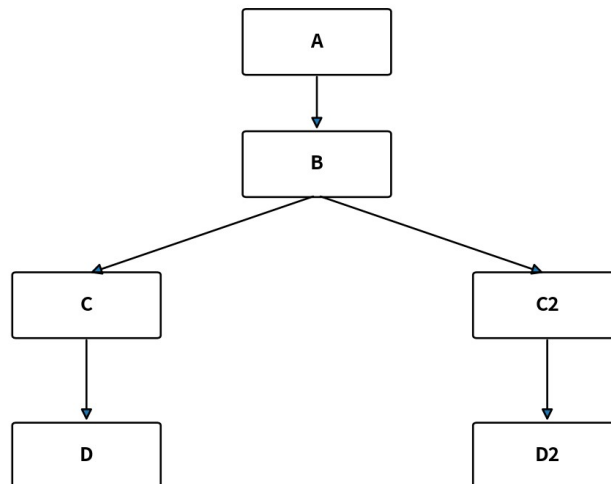
```
createAgentSession()
├─ resolve cwd / agentDir
├─ create ModelRuntime
├─ create SettingsManager
├─ create SessionManager
├─ ResourceLoader.reload()
├─ restore session context
├─ choose model / thinking
├─ select active tools
├─ new Agent(...)
└─ new AgentSession(...)
```

源码中默认激活的 built-in tools 是 read / bash / edit / write；allowlist、denylist、settings 与 custom tools 再对集合进行筛选。

Know-why Tool Registry 和 Active Tool Set 应该分离。注册一个能力不代表每一步都要把它暴露给模型。

12. SessionManager: Append-only Tree

Session = Append-only entries + parentId



Context projection: 选 leaf → 沿 parentId 回溯 → 应用 branch summary / compaction

图 4 Session entry 通过 parentId 形成树，而不是单一 messages 数组。

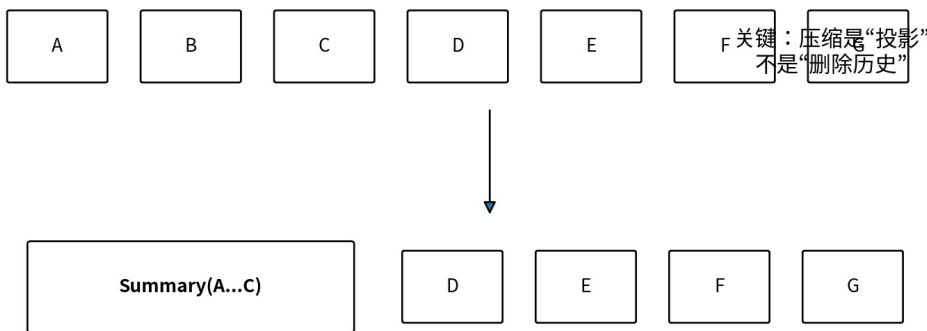
源码事实：Session 使用 JSONL append-only entries，每个普通 entry 都有 id、parentId、timestamp 与 type；选择一个 leaf 后，通过 parentId 回溯到 root 构造当前分支。

Entry 类型	用途	进入 LLM context?
message	user/assistant/toolResult	是
thinking_level_change	状态变更	影响 session settings，不直接作为 message
model_change	状态变更	影响恢复模型
compaction	旧上下文摘要	是，投影成 summary message
branch_summary	废弃/切换分支的信息摘要	是
custom	extension 持久化状态	否
custom_message	extension 注入上下文	是
label / session_info	展示与元数据	否

Know-how Persistent State 与 Model Context 必须是两个集合。Extension 需要保存状态时，不应该被迫把这些状态发送给 LLM。

13. Compaction: Context Projection, 不删除历史

Storage history (原始 JSONL 仍保留)



Active context = CompactionEntry + kept recent entries

图 5 Compaction 保留原始历史，只改变 active context projection。

Compaction 的核心不是 truncate JSONL，而是追加一个 CompactionEntry：包含摘要、firstKeptEntryId、tokensBefore 等；随后 buildContextEntries 只把摘要与需要保留的近端 entries 投影给模型。

Storage: A B C D E F G + CompactionEntry

Context: Summary(A..C) D E F G

触发与预算

当前默认 compaction settings 保留一定 reserve tokens，并保留一段 recent context。触发条件是 context 使用接近窗口上限，而不是等 provider 真正报 overflow 才处理。

Know-why Context window 不是“能塞满就塞满”的存储空间，而是下一轮 reasoning、tool calls 和 tool results 的工作内存。必须提前预留 headroom。

Cut point 的协议约束

Pi 不会把 toolResult 作为切割点，因为 toolResult 必须依赖其对应的 assistant tool call。Compaction 算法因此必须理解消息依赖，而不是只按 token 数从数组中间切一刀。

原则 任何 context trimming 都必须先满足 provider message protocol，再优化 token。合法性优先于压缩率。

14. Branch Summary 与 Context Memory

Session tree 带来一个新问题：用户从一条已探索分支跳到另一条分支时，旧分支可能包含有价值的发现。如果完全丢弃会损失知识；全部携带又会污染上下文。Branch summary 解决的是“保留对未来仍有价值的信息”。

```

explored branch
├─ failed approach
├─ discovered invariant
└─ key files
    ↓ summarize
BranchSummaryEntry
    ↓
new active branch context
  
```

Coding Agent Summary 为什么要追踪文件操作

Compaction 代码会从消息/工具调用中提取 readFiles 与 modifiedFiles，并把文件操作信息累积到 compaction details。原因是 Coding Agent 的长期工作状态并不只存在于自然语言里，环境对象同样重要。

Know-how 对 Coding/Research Agent，摘要应保留“环境状态”：已读文件、修改文件、关键 URL/文档、实验结果、未决问题。纯聊天摘要通常不足以继续工作。

15. AgentSessionRuntime: Session Replacement Lifecycle

`AgentSessionRuntime` 专门处理 new/resume/fork/import 等会话替换。它不是简单换 messages，而是替换一整套与 cwd 绑定的 services 与 runtime resources。

```

abort current run
  ↓ settle + persist
session_shutdown hooks
  ↓
host/UI teardown
  ↓
dispose old session
  ↓
recreate cwd-bound services
  ↓
create new AgentSession
  ↓
rebind host/UI
  
```

为什么 cwd 改变时需要重建 services？因为项目级 settings、AGENTS/context files、skills、extensions、tools 等都可能随工作目录变化。

Know-how Session switch 在 Agent 产品中往往等价于 Environment switch。把它设计成 lifecycle transaction，比直接替换 transcript 更安全。

16. Extension: Mechanism 与 Policy 分离

Extension 系统位于 Coding Agent 层，通过 AgentSession 将通用 before/after/context/lifecycle hook 转接给 ExtensionRunner。因此 agent-core 不需要认识 extension 概念。

```

Agent Core
  ↓ generic hooks
AgentSession
  ↓ adapter / runner ref
ExtensionRunner
  ↓
Extensions
  
```

稳定 callback + mutable indirection

Tool hooks 安装一次，但回调执行时读取当前 ExtensionRunner，而不是捕获旧实例。这样 extension reload 只需要替换 runner reference，不必重新配置 Agent。

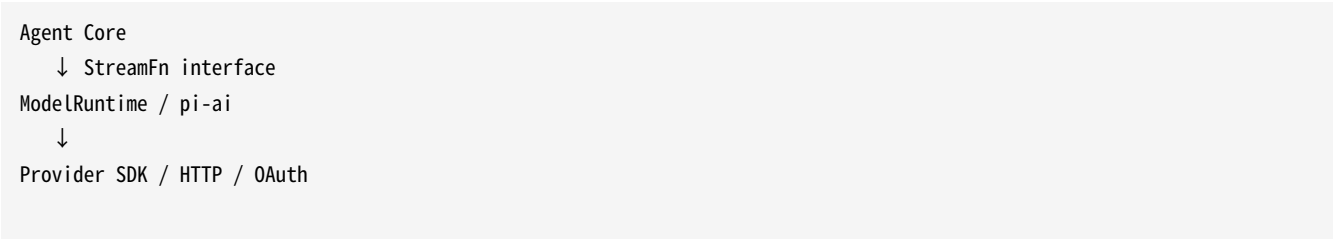
模式 Stable callback + mutable ref 是实现热重载和动态 policy 的实用技巧：保持 Agent hook 身份稳定，把实际策略通过 indirection 更新。

Extension 的合理边界

适合放 Extension	不适合放 Extension
Tool permission / policy	Agent Loop 基本控制流
Context injection	Provider 基础协议
Custom tools / commands	Session tree 的核心数据结构
Custom compaction behavior	通用 Abort / Event primitives

17. Provider Boundary 与 ModelRuntime

Agent Core 通过 StreamFn 等抽象调用模型，不直接依赖某个 provider SDK。Coding Agent 再由 ModelRuntime 处理 auth、model registry、provider-specific runtime 与 retry settings。



为什么统一 API 最难的不是 generate()

- 不同 provider 的 reasoning/thinking 表达不同。
- tool call / tool result 协议存在差异。
- usage 与 cache 字段不同。
- OAuth/API key 生命周期不同。
- 跨 provider 继续同一个 transcript 需要 message projection。

Know-how Provider abstraction 应统一 conversation continuity，而不仅是 prompt → text。Model、Message、ToolCall、ToolResult、Reasoning、Usage、FinishReason 都应有稳定内部表示。

18. 对照 learn-claude-code：从 s01 到 s10

学习阶段	Pi 源码对应	核心新增
s01 Agent Loop	agent-loop.ts / runLoop	LLM → Tool → Result → LLM
s02 Tool Use	executeToolCalls / prepare / finalize	validation、parallel/sequential、updates
s03 Permission	beforeToolCall	统一 policy gate
s04 Event Runtime	AgentEvent / EventStream	UI、telemetry 解耦

s05 Context Transform	transformContext / convertToLlm	Runtime state 与 model context 分离
s06 Session Persistence	SessionManager	append-only JSONL
s07 Branching	parentId / fork	conversation tree
s08 Compaction	compaction/*	context projection 与摘要
s09 Extensions	ExtensionRunner	mechanism/policy 分离
s10 Lifecycle	AgentSessionRuntime	session/environment replacement

如果把 learn-claude-code 当作最小教学链，Pi 可以视为后半程生产化设计的参考样本。

19. 哪些设计值得复制，哪些不该照抄

强烈建议复制

- Agent Core 与 Domain Harness 分离。
- Provider 通过 StreamFn/adapter 注入。
- Runtime Message 与 Provider Message 分离。
- Event-driven lifecycle。
- Tool execution pipeline + before/after hooks。
- Steer / Follow-up 两种队列语义。
- Session append-only tree。
- Compaction 作为 context projection，而不是破坏性删除。
- Extension runtime 通过稳定 hook 接口连接。

需要谨慎复制

部分	为什么谨慎	更适合的演进
AgentSession 职责较重	同时处理 retry/compaction/extensions/tools/persistence/model/bash	进一步拆 application services
JSONL + sync filesystem	local-first 极简，但不适合分布式多用户	抽象 SessionStore 后端
通用 bash 能力强	本地 coding 高效，但企业安全边界弱	sandbox + typed high-risk tools
单进程 extension	方便，但强隔离能力有限	process/worker isolation for untrusted extensions

20. 自研 Pi-like Runtime 的推荐重构

如果自研：继续拆分 AgentSession 的多重职责

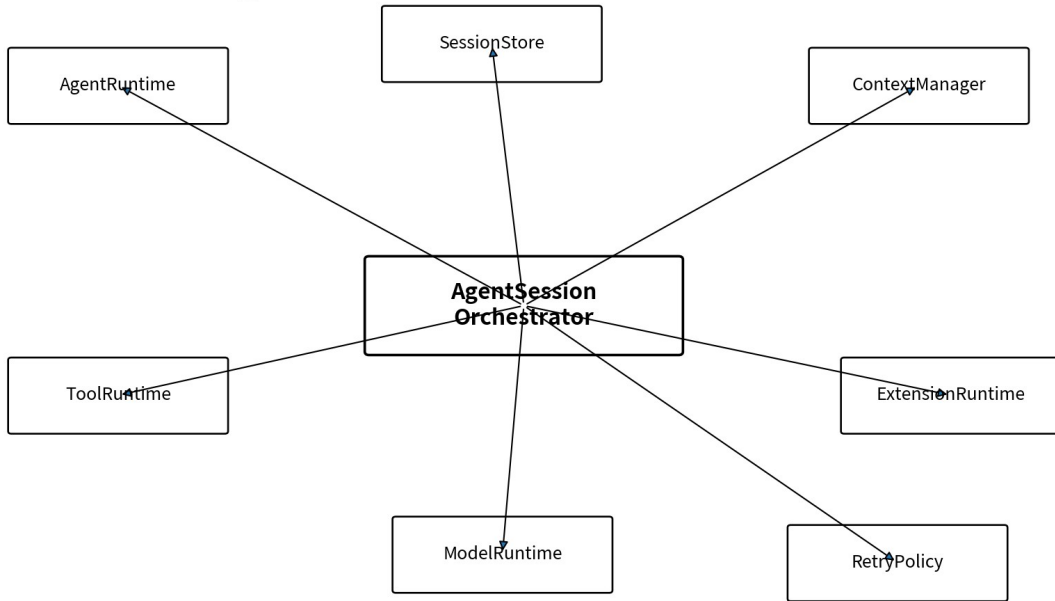


图 6 在 Pi 思想基础上，进一步把 AgentSession 拆成多个服务。

如果目标是长期维护的通用 Agent Runtime，可以保留 Pi 的核心思想，但把 Application Service 继续拆分。

```

src/
├── ai/
│   ├── model.ts
│   ├── message.ts
│   └── stream.ts
├── agent/
│   ├── loop.ts
│   ├── agent.ts
│   ├── events.ts
│   └── tool-runtime.ts
├── context/
│   ├── projection.ts
│   └── compaction.ts
├── session/
│   ├── entries.ts
│   └── store.ts
├── extension/
│   ├── hooks.ts
│   └── runner.ts
├── coding/
│   └── session-orchestrator.ts
  
```

推荐服务边界

服务	只负责什么
AgentRuntime	run、queue、abort、event
ToolRuntime	prepare/validate/policy/execute/finalize
ContextManager	projection、budget、compaction
SessionStore	append/read/tree/branch
ModelRuntime	model/auth/provider
ExtensionRuntime	hook registry 与 dispatch
RetryPolicy	错误分类、退避、最大尝试
AgentSession Orchestrator	组合上述服务，处理 domain workflow

最终原则 让 Agent Loop 保持机械、确定、可测试；把复杂性放在 Context、Session、Tool Policy、Extensions、Provider Runtime 与 Lifecycle 边界。

附录 A：源码阅读 Checklist

- ☐ 能画出 agentLoop / runLoop 的 outer-inner 两层循环。
- ☐ 能解释 steer 与 follow-up 的不同注入时机。
- ☐ 能解释 AgentMessage 与 Provider Message 为什么分离。
- ☐ 能指出 transformContext 与 convertToLlm 的职责边界。
- ☐ 能追踪一次 streaming assistant message 如何更新 state。
- ☐ 能列出 Tool pipeline 的 prepare/execute/finalize 三阶段。
- ☐ 能说明 beforeToolCall 为什么适合作为 permission gate。
- ☐ 能解释为什么截断 tool call 必须 fail closed。
- ☐ 能说明 sequential tool 如何影响整个 batch。
- ☐ 能解释 AgentSession 如何桥接 Agent hooks 与 ExtensionRunner。
- ☐ 能说明 prepareNextTurn 为什么要 late-bind tools/model/systemPrompt。
- ☐ 能画出 createAgentSession 的 composition root。
- ☐ 能解释 SessionEntry 的 parentId 如何形成 tree。
- ☐ 能区分 custom 与 custom_message。
- ☐ 能解释 compaction 为什么不删除原始 session history。
- ☐ 能说明 toolResult 为什么不能作为 cut point。
- ☐ 能解释 branch summary 的信息价值。
- ☐ 能画出 session replacement lifecycle。
- ☐ 能说明 Provider Runtime 为什么应该在 Agent Core 外。
- ☐ 能提出至少三项适合自研时继续重构的边界。

附录 B：关键函数索引

文件	关键函数/类型	阅读重点
packages/agent/src/agent-loop.ts	agentLoop / runAgentLoop / runLoop	主控制流
packages/agent/src/agent-loop.ts	streamAssistantResponse	Context → Provider → streaming state
packages/agent/src/agent-loop.ts	executeToolCalls*	并行/顺序 tool batch
packages/agent/src/agent-loop.ts	prepareToolCall	validation + before hook
packages/agent/src/agent-loop.ts	finalizeExecutedToolCall	after hook + logical result
packages/agent/src/agent.ts	Agent / PendingMessageQueue	stateful runtime、steer/followUp
packages/coding-agent/src/core/sdk.ts	createAgentSession	composition root
packages/coding-agent/src/core/agent-session.ts	AgentSession	Coding orchestration
packages/coding-agent/src/core/session-manager.ts	buildSessionContext / buildContextEntries	tree → active context
packages/coding-agent/src/core/compaction/compaction.ts	shouldCompact / findCutPoint / compact	context budgeting
packages/coding-agent/src/core/agent-session-runtime.ts	switchSession / newSession / fork	runtime replacement
packages/coding-agent/src/core/extensions/runner.ts	ExtensionRunner	event/hook dispatch

附录 C：最小教学版 Runtime 伪代码

下面不是 Pi 源码复制，而是根据其结构重写出的教学版伪代码，用于理解核心边界。

```
class AgentRuntime:
    state: AgentState
    steering_queue
    followup_queue

    async run(prompt):
        append(prompt)
        emit("agent_start")

        pending = drain_steering()
        while True:
            has_tools = True
            while has_tools or pending:
                emit("turn_start")
                inject(pending)

                ctx = context_manager.project(state.messages)
                msg = await model_runtime.stream(ctx)
                append(msg)

                calls = extract_tool_calls(msg)
                results = await tool_runtime.execute_batch(calls)
                append(results)
                emit("turn_end")

                refresh_dynamic_config()
                pending = drain_steering()
                has_tools = bool(calls)

            pending = drain_followups()
            if not pending:
                break

        emit("agent_end")
```

从伪代码继续演进的顺序

18. 先实现 EventStream 与稳定 AgentState。
19. 加入 Tool schema validation 与 before/after policy。
20. 加入 SessionStore，但先保持 append-only。
21. 加入 Context projection，而不是立即做复杂 Memory。
22. 实现 compaction，并确保 tool-call/tool-result 协议不被切断。
23. 加入 steer/follow-up queue。
24. 最后再做 Extension、Subagent、MCP、Planner 等上层能力。

结语：Pi 源码最应该带走的模型

```
Strong Model
+ Minimal Deterministic Loop
+ Explicit Tool Runtime
+ Context Projection
+ Append-only Session Tree
+ Extension Hooks
+ Real Environment Boundary
= Understandable Agent Harness
```

Pi 的价值不在于“工具少”这一个表面特征，而在于它把 Agent 系统中不同种类的状态与复杂性放到了不同层：执行状态归 Agent Core；上下文是投影；会话是持久历史树；工具有独立运行管线；策略通过 hook 注入；Provider 在边界之外；Coding workflow 由 AgentSession 编排。

如果要构建自己的 Agent Runtime，最重要的不是复制 Pi 的文件结构，而是复制这种边界意识：每一层都能明确回答“我为什么存在、我不负责什么”。

源码参考清单

本手册的源码事实主要基于以下仓库路径与对应快照。正文对源码做了结构化转述和伪代码抽象，没有大段复制源代码。

`packages/agent/src/agent-loop.ts`

`packages/agent/src/agent.ts`

`packages/agent/src/types.ts`

`packages/coding-agent/src/core/sdk.ts`

`packages/coding-agent/src/core/agent-session.ts`

`packages/coding-agent/src/core/agent-session-runtime.ts`

`packages/coding-agent/src/core/session-manager.ts`

`packages/coding-agent/src/core/compaction/compaction.ts`

`packages/coding-agent/src/core/compaction/branch-summarization.ts`

`packages/coding-agent/src/core/extensions/runner.ts`

`packages/coding-agent/src/core/extensions/types.ts`

`packages/coding-agent/src/core/resource-loader.ts`

`packages/coding-agent/src/core/system-prompt.ts`

`packages/coding-agent/src/core/tools/`

`packages/ai/`

Repository: [earendil-works/pi](https://github.com/earendil-works/pi)

Snapshot commit: [581d75a89cea21e50d6a26df840352f94427f633](https://github.com/earendil-works/pi/commit/581d75a89cea21e50d6a26df840352f94427f633)